DS: UNIT-1

1. a) Define Structure. Explain creation, accessing and initialization with suitable examples.
   b) Define Data structures. Explain Different Linear DS with representation.

A. a) **Structure**: A user defined data type that can be used to to group items of possibly different types into a single type.

→ 'struct' keyword is used to define a structure.

→ The structure contains members of different data types

Syntax:

```
struct structure_name
{   data_type 1  member 1;
    data_type 2  member 2;
    ....
    };
```

Creation: A structure is defined using 'struct' keyword, structure name and it contains its members. After that a variable to access the structure is created in two ways

i) **Create a structure variable seperately:**

Ex:

```
struct s
{ int x;
  };
struct s s1;  // s1 is the variable
```

ii) Creating a variable along with definition:

Ex:-
```
struct s
{ int x;
} s1, s2, s3,...; // Multiple variables
```

Initialization: It is done in 4 Methods

Example program:

```
#include <stdio.h>
#include <string.h>
 struct cartoon
{  char name[20];
   } s1 = {"Bheem"}; // Method 1
 int main()
{ struct cartoon s2 = {"Krish"}; // Method 2
   struct ~~student~~ cartoon s3;
   strcpy(s3.name, "Doraemon"); // Method 3
   struct cartoon s4, *p;
   p = &s4;  // Method 4
   scanf("%c", p->name);
   printf("%c\n%c\n%c\n%c", s1.name, s2.name,
                            s3.name, p->name);
  return 0;
}
```

Sample input: Nobita

Output: Bheem
Krish
Doraemon
Nobita

6) Data structure: A data structure is a storage that is used to store and organize data. It is a way of arranging data on a computer so that it can be accessed and updated efficiently.

→ Organizes, processes, stores data
→ Essential for nearly every software.
→ The data structures are classified into two types
  i) Linear Data Structure
  ii) Non-linear Data Structure

Linear Data Structures:

The Data structures in which Data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements, is called a linear data structure.

Ex:- Array, stack, Queue, linked list etc.

The linear Data Structures are divided into two types

i) Static Data structure:

→ Has fixed memory size
→ Easier to access elements

Example: Arrays

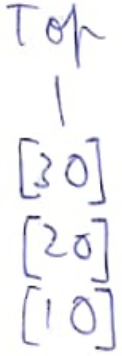
Index:	0	1	2	3	4
Array:	[10]	[20]	[30]	[40]	[50]

ii) Dynamic Data structure:

→ The size is not fixed
→ The data can be randomly updated during runtime
→ Efficient to space complexity of code.

Example:

i) Stack: Follows last in First out principle



ii) Queue: Follows First in First out principle

Front → [10][20][30][40] ← Rear

2. a, Explain Nested structures

b, Write a C program using typedef to define a new data type for structure.

A. a, Nested Structure: It is simply a structure within a structure. The structure declared inside a structure in the same way structure members are declared inside a structure. A structure can be nested in two ways

i, Separate nested structure:

```
#include <stdio.h>
#include <string.h>
 struct class
{   char class[20];
    int section;
    };
 struct student
{   char name[20];
    int rollno;
    struct class c;
    };
 int main()
{   struct student s;
    strcpy(s.name, "Lochan");
    s.rollno = 629;
    s.c.class = "DS";
    s.c.section = 2;
 printf("student : %c\n Rollno : %d\n Class : %c-%d",
        s.name, s.rollno, s.c.class, s.c.section);
    return 0;
}
```

Output: Student : Lochan
Rollno: 629
Class : DS-2

ii) Embedded Nested Structure:

```
#include <stdio.h>
#include <string.h>
 struct class
{   int section;
    struct student
    {   char name[20]
        int rollno;
        } s;

};
 int main()
{   struct class c;
    c.section = 2;
    c.s.name = "Lochan";
    c.s. rollno = 629;
   printf(" Class: DS - %d \n ", c.section);
   printf(" Name: %c \n", c.s. name );
   printf(" Rollno: %d ", c.s. rollno );
     return 0;
}
```

Output: Class : DS - 2
Name: Lochan
Rollno: 629

b, C program using typedef to define user data type for structure

```
#include <stdio.h>
#include <string.h>
 struct student
{  int rollno;
   char name[20];
   };
 int main()
{ typedef struct student s;
   ss1;
   s1.rollno = 629;
   s1.name = {"Lochan"};
   printf("Name: %c\n", s1.name);
   printf("Rollno: %d", s1.rollno);
   return 0;
}
```

Output:

Name: Lochan
Rollno: 629

3. a) Explain Array of structures with examples

b) Write a C program to calculate total and average marks of a student.

A. a) Array of Structures:

→ It is an array where each element is a structure.

→ It allows you to store several structures of the same type in a single array

Syntax:

```
struct struct_name arr_name[size];
```

→ Applications:

* storing the data of many no of employees.
* storing the data of students, etc.

Example program:

```
#include <stdio.h>
#include <string.h>
 struct student
{  char name[50];
   float marks;
   };
 int main()
{  struct student s[3]=
  {{"Lochan", 70.00},{"Rahul", 80.00},
    {"Shankar", 90.00}};
 for(int i=0; i<3; i++)
 {  printf("Student %d:\n", i+1);
```

```
    printf("Name: %s\n", s[i].name);
    printf("Marks: %.2f\n\n", s[i].marks);
  }
  return 0;
}
```

Output:

Student 1:
Name: Lochan
Marks: 70.00

Student 2:
Name: Rahul
Marks: 80.00

Student 3
Name: Shankar
Marks: 90.00

Write a program to calculate total and average marks of a student.

```
#include <stdio.h>
#include <string.h>
 struct student
{  char name[20];
   struct marks
   {  int math; sci, lang;
      } m;
 };
 int main()
{  struct student s;
   s.name = {"Lochan"};
   s.m.math = 99;
   s.m.sci  = 90;
   s.m.lang = 99;
   int total, average;
   total = (s.m.math + s.m.sci + s.m.lang);
   average = (total / 3);
   printf("student : %s\n", s.name);
   printf("Math: %d\n Science: %d\n Language: %d",
          s.m.math, s.m.sci, s.m.lang);
   printf("Total : %d \n", total);
   printf("Average : %d ", average);
   return 0;
}
```

Output:

Student : Lochan
Math: 99
Science: 90
Language: 99
Total : 288
Average : 96

4. a) Distinguish between. Array, structure and Union
b) Write a c program to declare an enum for days of week and print any given day.

A. a)

a)

	Array	Structure	Union
i)	An Array is a collection of elements of same data type stored in contiguous memory locations	A user defined data type that groups different data into a single entity	A user defined data type that allows different data types at same memory locations
ii)	Arrays are declared using a datatype keyword like int, etc	'Struct' keyword is used to define a structure	'Union' keyword is used to define a union.
iii)	The size of an array is fixed at time of declaration	The size is the sum of the sizes of all members	The size is equal to the size of the largest member.
iv)	The ~~memory~~ elements of an array are stored in contiguous memory locations	Each member is allocated unique storage area of the location	Memory allocated is shared by the individual members of union.
v)	No data overlap as members are independent	No data overlap as members are independent	Full data overlap as members share the same memory
vi)	Array elements are accessed using an index starting from 0.	In a structure the individual members can be accessed at a time.	In a union only one member can be accessed at a time.

b, Enum for days of week and print given day

```
#include <stdio.h>
enum weekday {Monday, Tuesday, Wednesday,
              Thursday, Friday, Saturday,
              Sunday};
int main()
{ enum weekday today, tomorrow;
  today = wednesday;
  tomorrow = Thursday;
  printf("Today : %d\n", today);
  printf("Tomorrow : %d\n", tomorrow);
  return 0;
}
```

Output :

Today = 3

Tomorrow = 4

5. a) Describe self referential structure with an example.
b) Write a C program to store and display display details of 5 students using array of structure.

A. a) Self referential structures:

→ the structures which have one or more pointers which point to the same type of structures as their members are called Self referential structures

→ Types of ~~self~~ self referential structures:

i) Self Referential Structure with single link

```
#include <stdio.h>
 struct ds
{ int rollno;
  char gen;
  struct ds *p;
  };
 int main()
{ struct ds s1 = {[illegible], 629, 'M'};
  s1.p = &s1;
  printf("Rollno = %d\n", s1.p->rollno);
  printf("Grender = %c\n", s1.p->gender);
  return 0;
}
```

Output:

Rollno = 629

Grender = M

Self Referential Structure with Multiple links

```
#include <stdio.h>
#include <stdlib.h>
struct student
{
  int rollno;
  char gender;
  struct student *next;
  struct student *prev;
};
int main()
{
  struct student s1, s2;
  s1.rollno = 1;
  s1.gender = 'M';
  s2.rollno = 2;
  s2.gender = 'F';
  s1.next = &s2;
  s1.prev = NULL;
  s2.next = NULL;
  s2.prev = &s1;
  printf("Student 1: %d\n Gender: %c\n",
         s1.rollno, s1.gender);
  printf("Student 2: %d\n Gender: %c\n",
         s2.rollno, s2.gender);
  return 0; }
```

Output:

```
Student 1:
Gender: M
Student 2:
Gender: F
```

Program to store and display details of 5 students;

```
struct students
{ int rollno;
  char name[20];
  int marks;
  };
#include <stdio.h>
#include <string.h>
 int main()
{  int n;
   scanf("%d", &n);
   struct students s[100];
   for(int i=0; i<=n; i++)
   { scanf("%d %s %d", &s[i].rollno, &s[i].name,
                        &s[i].marks); }

   for(int i=0; i<=n; i++)
   { printf("Student %d", i+1);
     printf("%d %s %d\n", s[i].rollno, s[i].name,
                          s[i].marks); }

   return 0; }
```

Sample input:

1 Pawan Kalyan 99
2 Chiranjeevi 98
3 Prabhas 97
4 Ram Charan 96
5 NTR 95

Output:

Student 1 : 1 Pawan Kalyan 99
Student 2 : 2 Chiranjeevi 98
Student 3 : 3 Prabhas 97
Student 4 : 4 Ram charan 96
Student 5 : 5 NTR 95

6. a) Write short note on i, Enum ii, typedef iii, Bitfields
b, Read and Display details of student rollno, name, marks.

A. a)

i) Enumeration: The user defined data type in C used to assign names to integral constants, the names make a program easy to read and maintain.

→ The keyword 'enum' is used to declare new enumerations types in C.

→ The assigning makes code more readable.

Syntax:

enum variable{ const1, const2, --- };

ii) typedef: The typedef is a keyword is used to provide existing data types with a new name. The C typedef keyword is used to redefine the name of already existing data types.

→ Used because datatypes become difficult to use programs

→ Behaves similarly to defining an alias for commands

Syntax: typedef existing_type new_type;

iii) Bitfields: The idea of bit-fields is to ~~used~~ use memory efficiently when we know that the value of a field or group of fields will never exceed a limit or is within a small range.

Syntax: struct structure_name
{ data_type member_name: width_of_bit-fields;
};

6, C program to read and display details of students

```
#include <stdio.h>
#include <string.h>
  struct student
{ char name[10];
   int rollno;
   int marks;
   };
  int main()
{ struct student s;
   scanf("%c\n%d\n%d\n", &s.name,
          &s.rollno, &s.marks );
   printf("%c\n%d\n%d\n", s.name, s.rollno,
                          s.marks );
   return 0;
}
```

Sample Input

```
Lochan
629
99
```

Output:

```
Lochan
629
99
```

7. a) Classify Data Structures with examples
b) Define a structure with bit fields

A. a) Data structure: A data structure is a storage that is used to store and organize data. It is a way of arranging data on a computer so that it can be accessed and updated efficiently.

→ It organizes, processes, stores data
→ Essential for nearly every software.
→ The data structure are classified into two types.
i) linear Data Structure
ii) Non-linear Data Structure

i) linear Data Structure: The Data structures in which data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent element, is called a linear data structure.

They are of two types

a) Static data structure:

→ Has fixed memory size
→ Easier to access elements

Ex: Arrays

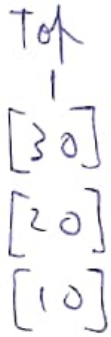
Index : 0 1 2 3 4

Array : [10] [20] [30] [40] [50]

b, Dynamic Data Structures:

→ The size is not fixed
→ The data can be randomly updated during runtime
→ Efficient to space complexity of code

Ex: Stack: Follows Last in First out principle



Queue: Follows First in First Out principle

Front → [10] [20] [30] [40] ← Rear

ii, Non Linear data structure: Data structures where data elements are not placed sequentially or linearly are called Non-Linear data structures. In Non linear data structure we can't traverse all elements in single run.

Ex:- Trees and Graphs

5, Define C program of structure with bit-fields

```
#include <stdio.h>
 struct date
{  int d:5;
   int m:4;
   int y;
   };
  int main()
{ printf("Size of date : %lu bytes \n", sizeof(struct date));
  struct date dt = {31,12,2014};
  printf("Date is %d/%d/%d", dt.d, dt.m, dt.y);
  return 0;
  }
```

Output:

Size of date: 8 bytes

Date is -1/-4/2014

8. a) Explain the concepts of structures and Functions
b) Write a C program to store and display details of a book (title, author, price)

Ans a) Structures and Functions: Using the structures and functions (or) passing a structure to the function can be done in 3 ways

i) Program with a structure and a function

// passing each item in a structure as a function argument

```
#include <stdio.h>
#include <string.h>
  struct student
{   int rollno;
    char name[20];
    };
  void display(int a, char name[20])
{   printf("Rollno: %d\n Student: %c", a, name); }
  int main()
{   struct student s1 = {629, "Lochan"};
    display(s1.rollno, s1.name);
    return 0;
  }
```

Output:

Rollno : 629

Student : Lochan

ii) Using Call by Value: Method

Example program:

```
#include <stdio.h>
#include <string.h>
  struct car
{  char name[30];
   int price;
   };
  Void print_car_info(struct car c)
{  printf("Name:%s\n",c.name);
   printf("Price :%od ", c.price);
   }
  int main()
{  struct car c = {"Lamborghini ", 10000000};
   print_car_info(c);
   return 0;
   }
```

Output:

```
Name : Lamborghini
Price : 10000000
```

III, Call by Reference Method:

```
#include <stdio.h>
#include <string.h>
 struct car
{  char name[30];
   int price;
   };
 Void display (struct car *c)
 int main()
{ printf("Name:%s\n",c->name);
  printf("Price :%d", c->price);
  }
 int main()
{  struct car c1 = {"Lamborghini", 10000000};
   display (&c1);
   return 0;
   }
```

Output:

Name : Lamborghini
Price : 10000000

b) C program to store details of a book.

```
#include <stdio.h>
#include <string.h>
 struct book
{  char title[20];
   char author[20];
   int price;
   };
  int main()
{  struct book b;
   b.title = "Ramayan";
   b.author= "Valmiki";
   b.price =20000000;
   printf("Details of Book:\n");
   printf("Title: %s\n", b.title);
   printf("Author: %s\n", b.author);
   printf("Price : %d\n", b.price);
   return 0;
}
```

Output:

```
Details of Book:
Title  : Ramayan
Author : Valmiki
Price  : 20000000
```

9. a) Define Union with examples
b) Write a C program to define a structure Employee

A) a) Union: The union is a user defined data type that can be contain elements of different data types just like structure.

→ Their only difference is that in unions, all the members are allocated the same memory locations

→ Only one member can be accessed at a time.

Syntax:

```
union _name
{ type1 member1;
  type2 member2;
  ----- };
```

Example program:

```
#include <stdio.h>
 union student
{ int rollno;
  char name[20];
  };
 int main()
{ union student s;
  s.rollno = 101;
  strcpy(s.name, "Ravi");
 printf("Rollno: %d \n", s.rollno);
 printf("Name: %s \n", s.name);
 return 0;
 }
```

Output:

```
Rollno: 101
Name: Ravi
```

b) C program to Define a structure with Employee details:

```
#include <stdio.h>
 struct Employee
{  char name[20];
   int empid;
   struct address
   {  char state[20];
      int pincode;
      } a;
};
 int main()
{  struct employee e = {"Lochan", 629, {"AP", 522001}};
   printf("Name: %s\n", e.name);
   printf("Emp ID: %d\n", e.empid);
   printf("State : %s\n", e.a.state);
   print("Pincode : %d", e.a.pincode);
   return 0;
 }
```

Output:

Name: Lochan

Emp ID: 629

State: AP

Pincode: 522001

DS: UNIT-2

1. Explain Quick Sort, Merge Sort, Radix Sort.

A. i, **Quick Sort:** It is a sorting technique used to sort the elements of an unsorted array which uses the 'divide and conquer' approach in its algorithm.

→ User a special element called the Pivot element

→ Partitions the given array around the Pivot elements and follows the following rules.

Rule-1:

Choose the pivot element

If the starting element is less than or equal to the pivot element the we do index increment

```
start++;
```

Rule-2:

If the end element is greater than the pivot element then we do index decrement.

```
end--;
```

Rule-3:

If the index of start element is less than index of end elements then we swap the start element and end element at respective indices.

Rule 4:

If 3rd rule is not satisfied then we swap end element with pivot element. Now the pivot is fixed and divide the array from pivot element and repeat process.

Tracing: Example array Taking start = 3
$a[5] = \{3, 0, 11, 25, 9\}$ pivot = 3
index : 0, 1, 2, 3, 4 end = 9

Rule	Condition	True / False	Operation	Result
1.	$3 \leq 3$	True	s++	1
	$0 \leq 3$	True	s++	2
	$11 \leq 3$	False	-	start index = 2 //
2.	$9 > 3$	True	e--	3
	$25 > 3$	True	e--	2
	$11 > 3$	True	e--	1
	$0 > 3$	False	-	end index = 1 //
3.	$2 < 1$	False	-	-
4.	Rule 3 is false	True	Swap pivot & new end elements	$3 \rightleftharpoons 0$

New array $a[5] = \{0, 3, 11, 25, 9\}$ Now
divide array from pivot element $a_1 = \{0\}$ & $a_2 = \{11, 25, 9\}$
Now a_2 needs to be sorted. index = 0 1 2
start element = 11 pivot = 11 end = 9

Rule	Condition	True / False	Operation	Result
1.	$11 \leq 11$	True	s++	1
	$25 \leq 11$	False	-	start index = 1 //
2.	$9 > 11$	False	-	end index = 2 //
3.	$1 < 2$	True	Swap elements at the indexes	$25 \rightleftharpoons 9$

New array $a_2 = \{11, 9, 25\}$

1.	$11 \leq 11$	True	s++	1
	$9 \leq 11$	True	s++	2
	$25 \leq 11$	False	-	start index = 2 //
2.	$25 > 11$	True	e--	
	$9 > 11$	False	-	end index = 1 //

3. $2 < 1$ False

4. Rule 3 is False — True — Swap pivot and new end elements

$9 \rightleftharpoons 11$

Sorted $a_1 = \{9, 11, 25\}$

∴ Sorted array $a[5] = \{0, 3, 9, 11, 25\}$

Program:

```
#include<stdio.h>
void quicksort(int a[], int start, int end)
{ int i, j, pivot, temp;
  if(start < end)
  {  pivot = a[start];
     i = start;
     j = end;
     while(i<j)
     {  while(a[i] <= pivot && i<end)
        { i++; }
        while(a[j] > pivot)
        { j--; }
     }
     if(i<j)
     { temp = a[i];
       a[i] = a[j];
       a[j] = temp;
     }
     temp = a[start];
     a[start] = a[j];
     a[j] = temp;
  quicksort(a, start, j-1);
  quicksort(a, j+1, end); }
}
```

```
int main()
{ int n, a[100];
  scanf("%d\n", &n);
  for(int i=0; i<n; i++)
  { scanf("%d", &a[i]); }
  quicksort(a, 0, n-1);
  printf("Sorting array: \n");
  for(int i=0; i<n; i++)
  { printf("%d", a[i]); }
  return 0;
}
```

Sample input:

5

3 0 11 25 9

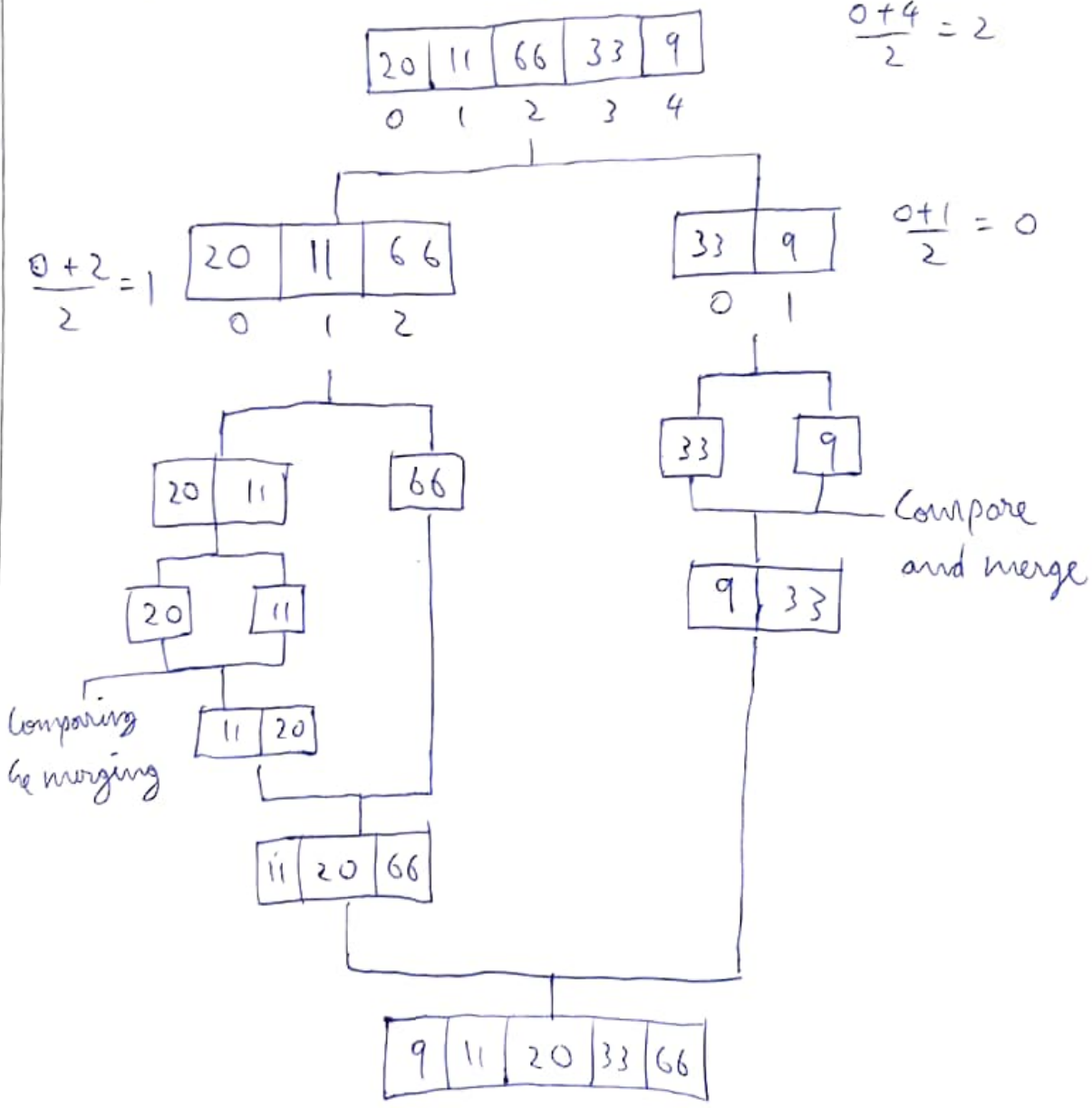
Output:

Sorted Array:

0 3 9 11 25

Merge sort: Merge sort is the sorting algorithm that uses the divide and conquer approach which divides the array into smaller sub array and merging the sorted sub arrays are then merged to form the sorted arrays.

Tracing: Example array $a[5] = \{20, 11, 66, 33, 9\}$



Sorted Array $a[5] = \{9, 11, 20, 33, 66\}$

Program:

```
#include <stdio.h>
 void merge(int a[ ], int lb, int mid, int ub)
{ int i=lb;
  int j = mid + 1;
  int k = lb;
  int b[100];
  while (i <= mid && j <= ub)
  { if(a[i] <= a[j])
    { b[k] = a[i];
      i++; }
    else
    { b[k] = a[j];
      j++; }
    k++;
  }
  while(i <= mid)
  { b[k] = a[i];
    i++;
    k++;
  }
  while(j <= ub)
  { b[k] = a[j];
    j++;
    k++;
  }
  for ( k= lb; k <= ub ; k++)
  { a[k] = b[k]; }
}
```

```
Void mergesort(int a[], int lb, int ub)
{
    if(lb<ub)
    {
        int mid = (lb+ub)/2;
        mergesort(a, lb, mid);
        mergesort(a, mid+1, ub);
        merge(a, lb, mid, ub);
    }
}

int main()
{
    int n, a[100];
    scanf("%d", &n);
    for(int i=0; i<n; i++)
    { scanf("%d", &a[i]); }
    mergesort(a, 0, n-1);
    printf("Sorted array:\n");
    for(int i=0; i<n; i++)
    { printf("%d", a[i]);
    return 0;
}
```

Sample Input

```
5
20 11 66 33 9
```

Output

```
Sorted array:
9 11 20 33 66
```

Radix Sort: The Radix Sort is a non-comparative sorting algorithm that sorts the numbers by processing individual digits. Starts from LSD to MSD using stable sorting algorithm.

→ After finding the maximum number counts the number of digits

→ The sorting will be completed in the no of steps = number of digits of maximum element.

Tracing: Example array a[5] = [31, 129, 6, 430, 22]

Max element = 430 No of digits = 3 = No of steps

Step 1: Units digit

Count[0] = 430

Count[1] = 031

Count[2] = 022

Count[3] =

Count[4] =

Count[5] =

Count[6] = 006

Count[7] =

Count[8] =

Count[9] = 129

→ a[5] = [430, 31, 22, 6, 129]

Steps 2: Next LSD

Count[0] = 006

Count[1] =

Count[2] = 022, 129

Count[3] = 430, 031

Count[4] =

Count[5] =

Count[6] =

Count[7] =

Count[8] =

Count[9] =

a[5] = [6, 22, 129, 430, 31]

Step 3: MSB

Count[0] = 006, 022, 031

Count[1] = 129

Count[2] =

Count[3] =

Count[4] = 430

Count[5] =

Count[6] =

Count[7] =

Count[8] =

Count[9] =

a[5] = [6, 22, 31, 129, 430]

After step-3

The Sorted array a[5] = [6, 22, 31, 129, 430]

Radix Sort: The Radix Sort is a non-comparative sorting algorithm that sorts the numbers by processing individual digits. Starts from LSD to MSD using stable sorting algorithm.

→ After finding the maximum number counts the number of digits

→ The sorting will be completed in the no of steps = number of digits of maximum element.

Tracing: Example array a[5] = [31, 129, 6, 430, 22]

Max element = 430 No of digits = 3 = No of steps

Step 1: Units digit

Count[0] = 430

Count[1] = 031

Count[2] = 022

Count[3] =

Count[4] =

Count[5] =

Count[6] = 006

Count[7] =

Count[8] =

Count[9] = 129

↳ a[5] = [430, 31, 22, 6, 129]

Steps 2: Next LSD

Count[0] = 006

Count[1] =

Count[2] = 022, 129

Count[3] = 430, 031

Count[4] =

Count[5] =

Count[6] =

Count[7] =

Count[8] =

Count[9] =

a[5] = [6, 22, 129, 430, 31] ↓

Step 3: MSB

Count[0] = 006, 022, 031

Count[1] = 129

Count[2] =

Count[3] =

Count[4] = 430

Count[5] =

Count[6] =

Count[7] =

Count[8] =

Count[9] =

a[5] = [6, 22, 31, 129, 430]

After step-3

The Sorted array a[5] = [6, 22, 31, 129, 430]

Program:

```
#include <stdio.h>
int getmax(int a[ ], int n)
{   int max = a[0], i;
    for (i = 1; i<n; i++)
    {   if (a[i] > max)
        {  max = a[i];}
    }
    return max;
}
void radixsort(int a[ ], int n)
{   int big, mod = 0, steps, count[10];
    int i, j, k, bucket[10][n], loc, div = 1;
    big = getmax(a, n);
    while (big > 0)
    {   mod++;
        big = big/10;
    }
    for (steps = 1; steps <= mod; steps++)
    {   for (j = 0; j<10; j++)
        {  count[j] = 0;}
        for (i = 0; i<n; i++)
        {  loc = (a[i]/div)% 10;
           bucket[loc][count[loc]++] = a[i];
        }
        k = 0;
```

```
    for(j=0;j<10;j++)
    { for(i=0;i<count[j];i++)
      { a[k++] = bucket[j][i];}
    }
    div = div * 10;
  }
}
int main()
{
  int a[100], n, i;
  scanf("%d", &n);
  for(i=0;i<n;i++)
  { scanf("%d", &a[i]); }
  radixsort(a,n);
  printf("Sorted Array by Radix sort : \n");
  for(i=0;i<n;i++)
  { printf("%d ", a[i]); }
  return 0;
}
```

Sample input

5

31 129 6 430 22

Output:

Sorted Array by Radix sort:

6 22 31 129 430

2. Define Algorithm. Explain efficiency of an algorithm with examples.

A. Algorithm: An algorithm is a step-by-step procedure used to solve a problem or perform a task in a finite no of steps.

Efficiency of an Algorithm:

The efficiency of an algorithm explains how fast and how much memory an algorithm to perform the task. Its measured by Time and space complexities

i, Time complexity: The measure of how much time an algorithm takes to run depending on the size of input.

→ Generally represented using 'O' notation.

Time complexity	Function	Name
	$O(1)$	- Constant time
	$O(\log n)$	- Logarithmic
	$O(n)$	- linear
	$O(n \log n)$	- log linear
	$O(n^2)$	- Quadratic
	$O(n^3)$	- Cubic
	$O(2^n)$	- Exponential

Example program

```
for(int i=1; i<=n; i++)
{ printf("%d", i); }
```

Note
- Loop runs n times
- Each time takes 1 step
- Total steps = n

∴ Time complexity = O(n)

Space complexity: It is the measure of how much memory an algorithm needs to run depending on the size of input(n).

Space complexity		Name
O(1)	-	Constant space
O(n)	-	linear space
$O(n^2)$	-	Quadratic space
O(log n)	-	logarithmic space

Example:

```
int a, b, c;
c = a + b;
```

Note
- Only 3 variables
- Memory does not change with n

Space complexity = O(1)

3. Explain Asymptotic Notations with Graphs.

A. Asymptotic Notations:

→ Asymptotic notations are mathematical tools used to analyze the performance of algorithms by understanding how their efficiency changes as input size grows.

→ Provides a concise way to express behaviour of an algorithm.

→ This analysis shows for comparison of algorithms.

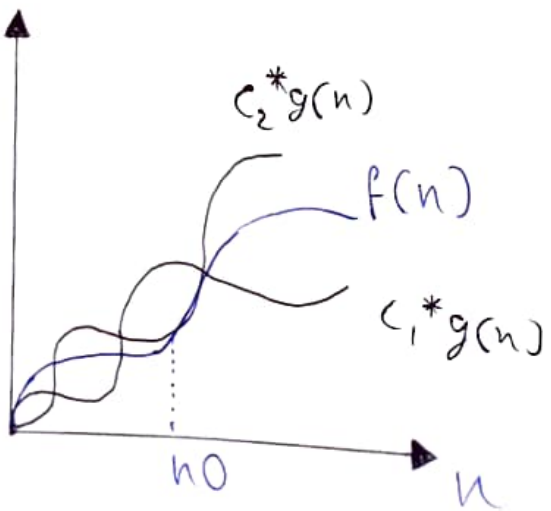
→ Examines their performance characteristics as the input size varies.

→ There are mainly 3 asymptotic notations:

1. Theta Notation (Θ):

- Represents the upper and lower bounds of running time of an algorithm.
- Used for analysing average-case complexity
- Let g & f be function from set of natural numbers. The f is said to be Θ(g), if $c1, c2 > 0$ and $c1 * g(n) \leq f(n) \leq c2 * g(n)$ for all $n \geq n0$

Graph:



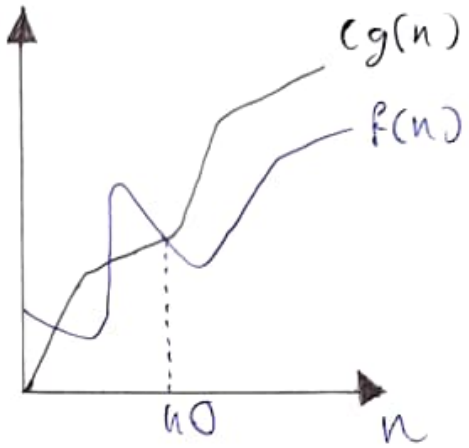
ii) Big O-Notation:

→ The Big-O notation represents the upper bound of the running time of an algorithm.

→ therefore, it gives the ~~worst~~ worst-case complexity

→ If f(n) describes time complexity then f(n) is O(g) if $0 \leq f(n) \leq cg(n)$ for all $n \geq n0$ & constant c.

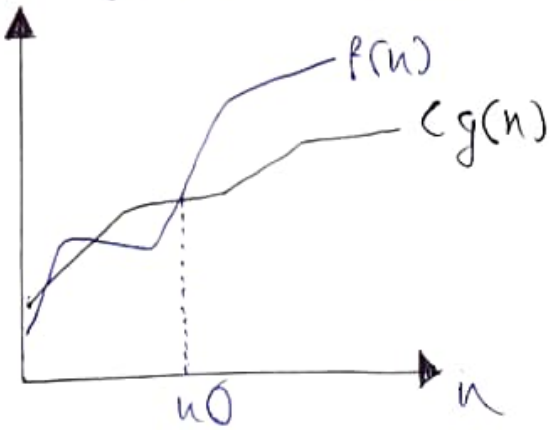
Graph:



iii, Omega Notation (Ω)

- Omega notation represents the lower bound of the running time of an algorithm.
- It provides the best case complexity of algorithm
- Let g & f be functions from natural number set. The f is $\Omega(g)$, if constant $c > 0$ and $c^* g(n) \leq f(n)$ for all $n \geq n0$

Graph:



4. Define ADT. Explain different ADTs with operations

A. An Abstract Data Type (ADT) is a conceptual model that defines a set of operations and behaviours for a data structure, without specifying how these operations are implemented or how data is organized in memory.

Types of ADTs:

i, List ADT: It is a sequential collection of elements. The list ADT needs to have following functions

a) get(): Return any element from list

b) insert(): Insert element at any position of list

c) remove(): Removes first element of list

d) removeat(): Remove any element of list

e) replace(): Replace an element at any position

f) size(): Return the size of list

ii) Stack ADT: The linear data structure that follows last in, first out. The functions it follows.

a) push(): Insert element at one end called the top

b) pop(): Remove and return element at the top

c) peek(): Only returns element at the top.

d) siz(): Returns the size of stack

e) isEmpty & is Full(): Returns if the stack is empty or full

iii) Queue ADT: It is a linear Data structure that follows the First in, First out. Its functions are

a) enqueue(): Insert an element at the end

b) dequeue(): Remove and return the first element

c) peek(): Only Returns the element of queue

d) size(): Return the size of queue

e) isEmpty(): Return true if queue is empty, false if other wise.

→ ADT's have some advantages

i) Encapsulation to manage and modify DS.

ii) Abstraction to simplify program and reduce errors

iii) Data Structure Independence

iv) Information Hiding

v) Modularity to form more complex data structures